

BovinNet

Sistema de gestión ganadera — aplicación web

Evidencia de desempeño

GA8-220501096-AA1-EV01

Desarrollar software a partir de la integración de sus módulos componentes

Programa de formación: Análisis y Desarrollo de Software

Proyecto: BovinNet — Sistema de gestión ganadera

Formato de entrega: PDF + código fuente (repositorio Git)

Documento asociado: GFPI-F-135 V01

1. Introducción

BovinNet es una aplicación web para el registro y seguimiento de un hato bovino. Permite llevar el inventario de animales, registrar nacimientos, dar de baja animales por muerte u otras causas, y llevar el historial de vacunación de cada bovino. El sistema nació en una fase temprana de formación como un conjunto de páginas estáticas de referencia (maquetas en HTML) y ha evolucionado, a lo largo del proceso formativo, hacia una aplicación funcional con una arquitectura por capas, persistencia en base de datos y un módulo de seguridad con autenticación y control de acceso por rol.

Este documento corresponde a la evidencia de desempeño GA8-220501096-AA1-EV01 y describe la codificación e integración de los módulos del sistema: los requerimientos considerados, los diagramas de diseño (clases, paquetes, componentes y mapa de navegación), la arquitectura por capas, los mecanismos de seguridad aplicados, las librerías y frameworks utilizados, y la configuración de los ambientes de desarrollo y pruebas. Dado que el levantamiento de requisitos e historias de usuario del proyecto se realizó en una etapa anterior del proceso formativo, y la solución evolucionó desde entonces, dichos artefactos se presentan aquí actualizados para reflejar fielmente el alcance realmente implementado.

2. Alcance del proyecto

El alcance de la versión actual del software comprende los siguientes módulos funcionales:

- Autenticación de usuarios (inicio y cierre de sesión) y cambio de contraseña.
- Control de acceso por rol (Administrador, Veterinario, Operario).
- Gestión de bovinos: registro y listado del inventario de animales.
- Nacimientos: registro de crías, con vínculo opcional a la madre y creación automática del bovino.
- Muertes / bajas: registro de la baja de un bovino con causa y observaciones.
- Vacunación: registro de la aplicación de vacunas y su historial por bovino o por fecha.
- Panel principal: indicadores y actividad reciente calculados en tiempo real a partir de la base de datos (sin valores de ejemplo).

Quedan fuera del alcance de esta entrega el módulo de Reportes (se mantiene como maqueta de referencia para una futura iteración) y la administración de usuarios y lotes desde la propia aplicación, que por ahora se gestionan directamente en la base de datos.

3. Metodología de desarrollo

El proyecto se desarrolló siguiendo un enfoque iterativo e incremental, ajustado al ritmo del proceso de formación: cada evidencia entregada correspondió a un incremento funcional verificable sobre el incremento anterior, en lugar de intentar construir el sistema completo desde el inicio. En términos prácticos, se aplicaron prácticas propias de metodologías ágiles (inspiradas en Scrum):

- **Incrementos cortos:** cada módulo (bovinos, nacimientos, bajas, vacunación, seguridad) se entregó y se probó de forma funcional antes de iniciar el siguiente.
- **Retroalimentación continua:** cada incremento se validó ejecutando la aplicación en un entorno local (XAMPP) y corrigiendo los defectos encontrados antes de avanzar.

- **Priorización del valor funcional:** primero se aseguró el registro correcto de datos (persistencia), luego la navegación completa entre módulos, y finalmente la seguridad transversal (autenticación, autorización y protección de formularios).
- **Diseño evolutivo:** la arquitectura por capas (Modelo, Repositorio, Controlador, Vista) se introdujo desde el primer módulo funcional y se reutilizó como plantilla para los módulos siguientes, lo que redujo el tiempo de codificación de cada nuevo incremento.

4. Requerimientos del sistema

4.1 Requerimientos funcionales

Código	Descripción	Módulo
RF-01	El sistema debe permitir iniciar sesión con correo y contraseña, y cerrar sesión.	Seguridad
RF-02	El sistema debe restringir el acceso a cada módulo según el rol del usuario autenticado.	Seguridad
RF-03	El usuario autenticado debe poder cambiar su propia contraseña.	Seguridad
RF-04	El sistema debe permitir registrar un bovino con arete, nombre, sexo, peso, fecha de nacimiento y lote, validando que el arete sea único.	Bovinos
RF-05	El sistema debe listar los bovinos registrados.	Bovinos
RF-06	El sistema debe permitir registrar el nacimiento de una cría, indicando opcionalmente la madre, y crear automáticamente el bovino correspondiente.	Nacimientos
RF-07	El sistema debe listar el historial de nacimientos y permitir filtrarlo por fecha.	Nacimientos
RF-08	El sistema debe permitir registrar la baja de un bovino activo, indicando causa y observaciones, con confirmación previa del usuario.	Muertes/Bajas
RF-09	El sistema debe listar el historial de bajas y permitir filtrarlo por causa.	Muertes/Bajas
RF-10	El sistema debe permitir registrar la vacunación de un bovino activo, indicando tipo de vacuna y fecha de aplicación.	Vacunación
RF-11	El sistema debe listar el historial de vacunación y permitir filtrarlo por fecha.	Vacunación
RF-12	Un bovino dado de baja no debe aparecer disponible para nuevas vacunaciones ni como madre en nuevos nacimientos.	Transversal
RF-13	El panel principal debe mostrar indicadores reales calculados a partir de la base de datos (bovinos activos, nacimientos, bajas y vacunaciones del mes) y la actividad reciente del sistema, sin datos de ejemplo.	Panel principal

4.2 Requerimientos no funcionales

Código	Descripción
RNF-01	La aplicación debe ejecutarse en un entorno LAMP/WAMP estándar (Apache, PHP 8, MySQL) sin dependencias externas de pago.
RNF-02	Toda consulta a la base de datos debe usar sentencias preparadas (PDO) para prevenir inyección SQL.
RNF-03	Las contraseñas deben almacenarse cifradas con un algoritmo de hash robusto (bcrypt), nunca en texto plano.
RNF-04	La sesión del usuario debe cerrarse automáticamente tras 30 minutos de inactividad.
RNF-05	Los formularios que modifican datos deben protegerse contra ataques de tipo CSRF.
RNF-06	La interfaz debe mantener un diseño visual consistente entre todos los módulos.
RNF-07	Los archivos de configuración con credenciales no deben quedar expuestos ni versionados en el repositorio de código.

5. Historias de usuario

Las historias de usuario levantadas en una fase temprana del proceso de formación no reflejaban con exactitud el alcance final implementado, por lo que se presentan a continuación reformuladas a partir del sistema realmente construido.

HU-01 — Inicio de sesión

Como usuario del sistema, **quiero** iniciar sesión con mi correo y contraseña, **para** acceder únicamente a los módulos que corresponden a mi rol.

Criterios de aceptación:

- Si el correo o la contraseña son incorrectos, se muestra un único mensaje genérico de error.
- Tras 5 intentos fallidos en 15 minutos con el mismo correo, el acceso se bloquea temporalmente.
- Al iniciar sesión correctamente, se redirige al panel principal.

HU-02 — Consultar panel principal

Como usuario autenticado, **quiero** ver un resumen del estado actual del hato al ingresar al sistema, **para** tener una visión rápida de la operación sin tener que revisar cada módulo por separado.

Criterios de aceptación:

- Los indicadores (bovinos activos, nacimientos, bajas y vacunaciones del mes) se calculan a partir de los datos reales almacenados, no de valores de ejemplo.
- La variación mensual se compara contra el mes calendario anterior.
- La actividad reciente combina, en una sola línea de tiempo, los últimos nacimientos, bajas y vacunaciones registrados.

HU-03 — Registrar bovino

Como operario de la finca, **quiero** registrar un nuevo bovino con sus datos básicos, **para** mantener actualizado el inventario del hato.

Criterios de aceptación:

- El arete es obligatorio y no se permite repetir uno ya registrado.
- El peso y la fecha de nacimiento son opcionales, pero si se ingresan deben ser válidos.
- El bovino registrado aparece inmediatamente en el listado.

HU-04 — Registrar nacimiento

Como operario de la finca, **quiero** registrar el nacimiento de una cría e indicar quién es la madre, **para** llevar la trazabilidad genealógica del hato.

Criterios de aceptación:

- Si se indica una madre, la fecha de nacimiento de la cría debe ser posterior a la de la madre.
- Solo pueden seleccionarse como madre hembras activas (no dadas de baja).
- Al guardar, se crea automáticamente el bovino correspondiente a la cría.

HU-05 — Registrar baja

Como veterinario de la finca, **quiero** registrar la baja de un bovino indicando la causa, **para** mantener el historial sanitario y evitar operar sobre animales que ya no están activos.

Criterios de aceptación:

- El sistema solicita confirmación explícita antes de guardar la baja.
- Un bovino dado de baja deja de estar disponible para nuevas vacunaciones o como madre.
- La causa debe seleccionarse de una lista predefinida; "Otra" exige una observación.

HU-06 — Registrar vacunación

Como veterinario de la finca, **quiero** registrar la vacuna aplicada a un bovino y la fecha, **para** llevar el control sanitario del hato y consultar el historial cuando lo necesite.

Criterios de aceptación:

- El tipo de vacuna puede elegirse de una lista de sugerencias o escribirse libremente.
- La fecha de aplicación no puede ser futura ni anterior al nacimiento del bovino.
- El historial puede filtrarse por fecha de aplicación.

HU-07 — Cambiar contraseña

Como usuario autenticado, **quiero** cambiar mi propia contraseña, **para** proteger mi cuenta, especialmente tras usar una contraseña temporal.

Criterios de aceptación:

- Se exige la contraseña actual correcta antes de permitir el cambio.
- La nueva contraseña debe tener mínimo 8 caracteres y combinar letras y números.
- La nueva contraseña no puede ser igual a la actual.

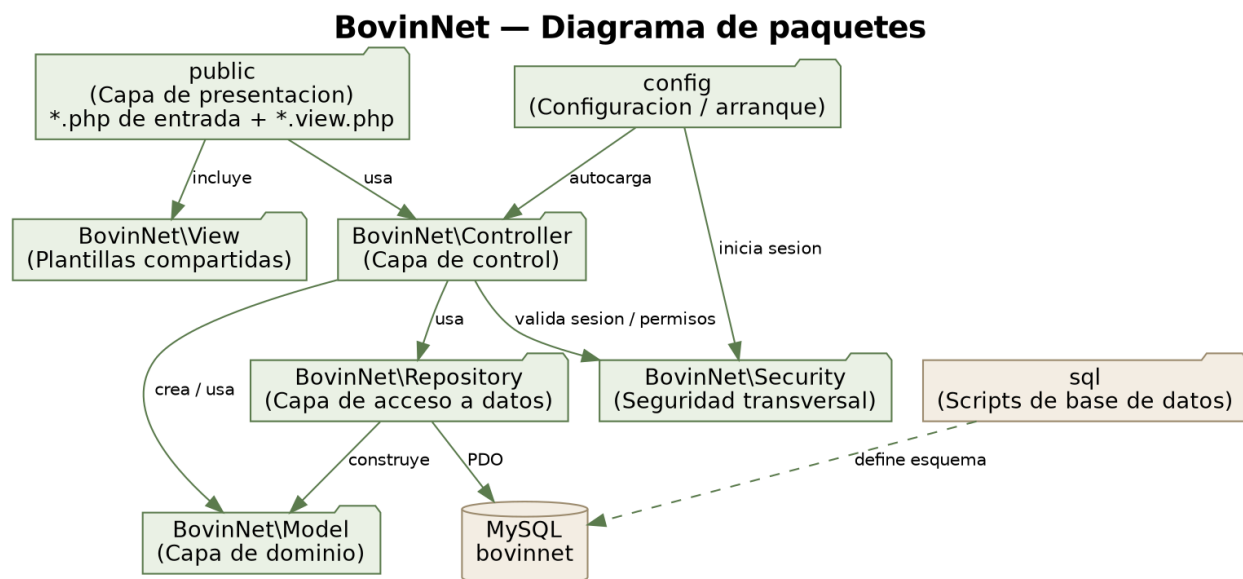
6. Arquitectura del software

6.1 Capas de la aplicación

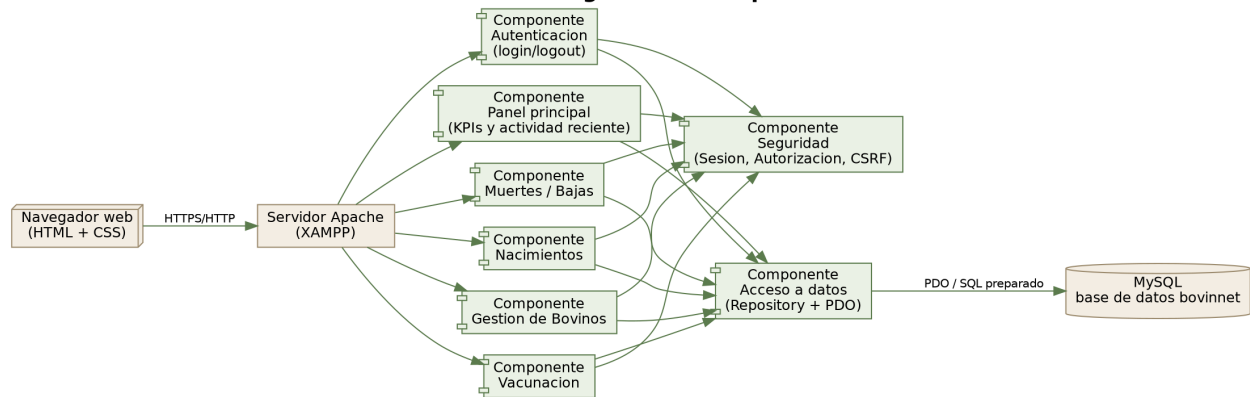
El sistema se organiza en cuatro capas, siguiendo una variante del patrón Modelo-Vista-Controlador con una capa adicional de acceso a datos (Repository), más un eje transversal de seguridad:

Capa	Responsabilidad	Ubicación en el código
Presentación (Vista)	Maquetación HTML/CSS y renderizado de datos ya validados. No contiene lógica de negocio ni acceso a datos.	public/*.view.php, src/View/
Control	Recibe la petición HTTP, valida la entrada del usuario, coordina el uso del modelo y del repositorio, y decide qué vista mostrar.	src/Controller/
Dominio (Modelo)	Representa las entidades del negocio y sus reglas propias (por ejemplo, la validez del arete de un bovino).	src/Model/
Acceso a datos (Repository)	Encapsula las sentencias SQL preparadas contra MySQL mediante PDO. Es la única capa que conoce el esquema de la base de datos.	src/Repository/
Seguridad (transversal)	Sesión, autenticación, autorización por rol y protección CSRF. Es consultada por la capa de control antes de ejecutar cualquier operación.	src/Security/

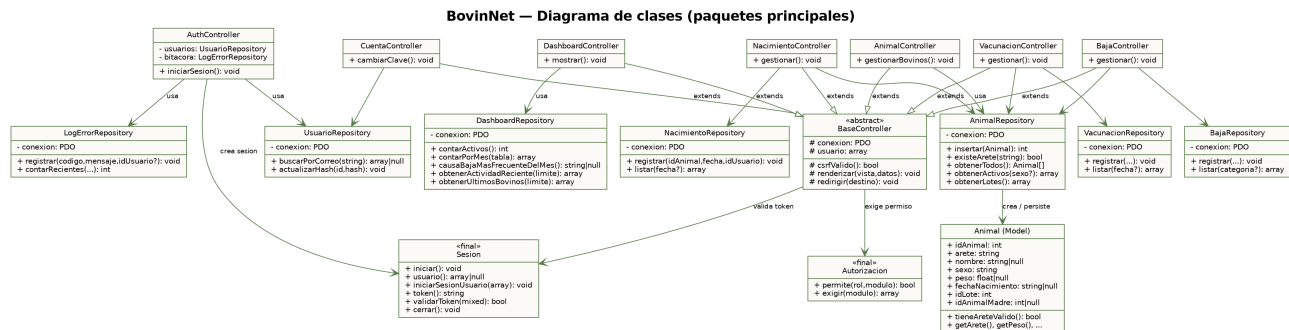
6.2 Diagrama de paquetes



6.3 Diagrama de componentes

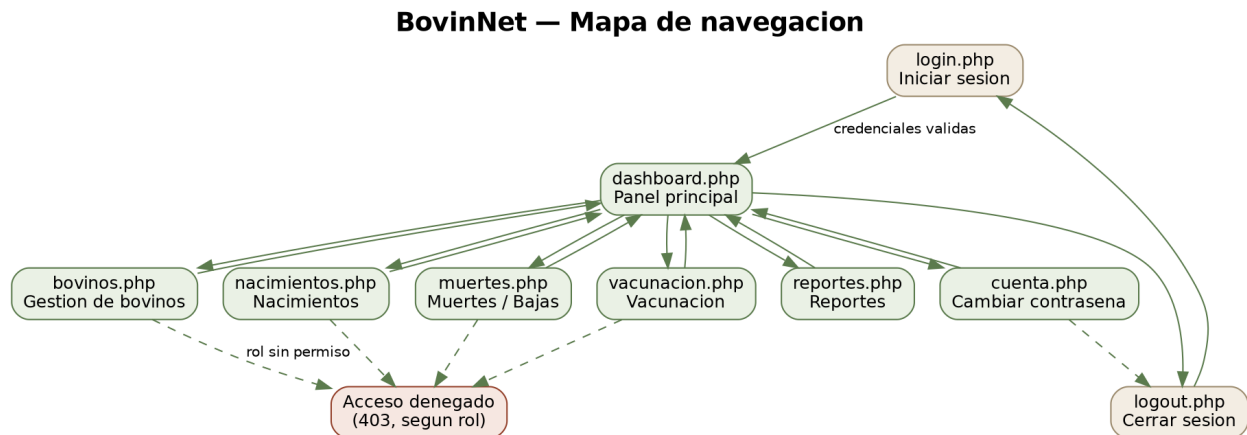
BovinNet — Diagrama de componentes

6.4 Diagrama de clases



El diagrama muestra las clases de los paquetes Model, Repository, Controller y Security. Todos los controladores de módulo heredan de BaseController, que centraliza la validación de sesión, el token CSRF y el renderizado de vistas.

6.5 Mapa de navegación



7. Patrones de diseño aplicados

Patrón	Dónde se aplica	Motivo
MVC (Modelo-Vista-Controlador)	Estructura general de la aplicación.	Separar la lógica de negocio de la presentación y facilitar el mantenimiento.
Repository	AnimalRepository, NacimientoRepository, BajaRepository, VacunacionRepository, UsuarioRepository, LogErrorRepository.	Aislar el acceso a datos (SQL/PDO) del resto de la aplicación; permite cambiar el motor de base de datos sin afectar los controladores.
Template Method (plantilla)	BaseController y src/View/layout_inicio.php / layout_fin.php.	Reutilizar el flujo común (verificar sesión, permisos, encabezado y menú) entre todos los módulos.
Front Controller (por página)	Cada archivo public/*.php actúa como punto de entrada único que delega en un controlador.	Mantener una URL simple y predecible por módulo.
Singleton implícito de sesión	BovinNet\Security\Sesion, con métodos estáticos sobre \$_SESSION.	Garantizar un único punto de acceso y de reglas para la sesión (expiración, token CSRF).

8. Librerías y frameworks por capa

El proyecto no utiliza un framework de aplicación completo (como Laravel o Symfony): se construyó en PHP nativo estructurado por capas, usando exclusivamente extensiones y herramientas incluidas en la distribución estándar de PHP y en XAMPP. Esta decisión responde al alcance educativo del proyecto y facilita revisar cada mecanismo (sesión, hashing, PDO) de forma explícita en el código.

Capa	Lenguaje / tecnología	Librería o extensión
Presentación	HTML5, CSS3	Hoja de estilos propia (public/css/style.css); sin frameworks CSS externos.
Control / Dominio	PHP 8 (tipado estricto, declare(strict_types=1))	PHP estándar; autocarga de clases propia (PSR-4 simplificado) en config/bootstrap.php.
Seguridad	PHP 8	Extensión password_hash/password_verify (bcrypt); función random_bytes para tokens CSRF y de sesión.
Acceso a datos	PHP 8 + SQL	Extensión PDO (PDO_MYSQL) con sentencias preparadas.
Base de datos	MySQL / MariaDB (motor de XAMPP)	Scripts SQL propios (sql/bovinnet.sql, sql/datos_iniciales.sql).
Servidor	Apache (XAMPP)	Módulo mod_rewrite no requerido; control de acceso a carpetas mediante archivos .htaccess.

9. Mecanismos de seguridad

Mecanismo	Implementación
Autenticación	Login con correo y contraseña; verificación con <code>password_verify()</code> contra un hash bcrypt almacenado en la tabla usuario.
Cifrado de contraseñas	<code>password_hash()</code> con el algoritmo por defecto de PHP (bcrypt). Se recalcula el hash automáticamente si el algoritmo por defecto cambia (<code>password_needs_rehash</code>).
Control de fuerza bruta	Bitácora de intentos fallidos en la tabla <code>log_error</code> ; bloqueo temporal tras 5 intentos fallidos en 15 minutos para el mismo correo.
Gestión de sesión	Cookie de sesión con <code>httponly</code> y <code>samesite=Lax</code> ; regeneración del identificador de sesión al iniciar sesión (previene fijación de sesión); expiración a los 30 minutos de inactividad.
Autorización por rol	Matriz de permisos por rol y módulo (<code>BovinNet\Security\Autorizacion</code>); toda página protegida exige sesión activa y permiso explícito, o responde 403.
Protección CSRF	Token aleatorio de 32 bytes por sesión, incluido en todo formulario POST (incluido el cierre de sesión) y validado con <code>hash_equals()</code> .
Prevención de inyección SQL	Todas las consultas usan sentencias preparadas de PDO con parámetros nombrados; ninguna concatena directamente datos del usuario en el SQL.
Prevención de XSS	Toda salida de datos hacia HTML pasa por la función <code>e()</code> (<code>htmlspecialchars</code> con <code>ENT_QUOTES</code> y codificación UTF-8).
Cabeceras HTTP de seguridad	X-Frame-Options: DENY, X-Content-Type-Options: nosniff y Referrer-Policy: same-origin en cada respuesta.
Protección de archivos del servidor	Reglas <code>.htaccess</code> que deniegan el acceso web directo a <code>config/</code> , <code>src/</code> , <code>sql/</code> y a las plantillas <code>*.view.php</code> .
Gestión de credenciales	<code>config/db.php</code> se excluye del repositorio (<code>.gitignore</code>); se entrega <code>config/db.example.php</code> como plantilla sin datos sensibles.

10. Codificación de los módulos

A continuación se describe brevemente cada módulo codificado y se referencia el archivo principal donde reside su lógica de control. El código fuente completo se entrega junto con este documento.

10.1 Panel principal

Controlador: `src/Controller/DashboardController.php`

Obtiene de `DashboardRepository` los indicadores del mes en curso (nacimientos, bajas y vacunaciones, comparados con el mes anterior), el conteo de bovinos activos, la causa de baja más frecuente del mes y la actividad reciente combinando las tres tablas de eventos. Ningún valor del panel está escrito de forma fija: todos se calculan con consultas SQL sobre la base de datos en el momento de la petición.

10.2 Autenticación y sesión

Controlador: `src/Controller/AuthController.php`

Valida las credenciales, aplica el bloqueo por intentos fallidos, verifica la contraseña con `password_verify()` y, si es correcta, delega en `Sesion::iniciarSesionUsuario()` para regenerar el identificador de sesión y emitir el token CSRF.

10.3 Gestión de bovinos

Controlador: `src/Controller/AnimalController.php`

Valida los datos del formulario (arete único, sexo, peso, fecha, lote), construye un objeto `Animal` y lo persiste mediante `AnimalRepository::registrarNacimiento()`. Extiende `BaseController`, por lo que hereda la verificación de sesión, permisos y token CSRF.

10.4 Nacimientos

Controlador: `src/Controller/NacimientoController.php`

Valida la fecha de nacimiento contra la fecha de la madre (si se indicó una), crea el bovino hijo y su registro de nacimiento dentro de una transacción de base de datos, de modo que ambas inserciones se confirman o se revierten juntas.

10.5 Muertes / bajas

Controlador: `src/Controller/BajaController.php`

Exige confirmación explícita del usuario antes de registrar la baja, valida la causa contra una lista cerrada de categorías y excluye del listado de "animales activos" a los ya dados de baja, lo que impide vacunarlos o usarlos como madre en un nacimiento posterior.

10.6 Vacunación

Controlador: `src/Controller/VacunacionController.php`

Registra la vacuna aplicada a un bovino activo, valida que la fecha de aplicación no sea futura ni anterior al nacimiento del animal, y permite filtrar el historial por fecha.

10.7 Cambio de contraseña

Controlador: `src/Controller/CuentaController.php`

Exige la contraseña actual correcta, valida la robustez mínima de la nueva contraseña (8 caracteres, letras y números) y regenera el identificador de sesión tras el cambio.

11. Pruebas

11.1 Pruebas funcionales realizadas

Cada módulo se probó de forma manual y funcional sobre un entorno local (XAMPP), verificando los siguientes escenarios:

Módulo	Escenario probado	Resultado esperado
Login	Correo y contraseña correctos.	Redirige al panel principal.
Panel principal	Registrar un nacimiento, una baja y una vacunación, y volver a abrir el panel.	Los conteos del mes y la actividad reciente reflejan los nuevos registros de inmediato.
Login	Contraseña incorrecta, 5 veces seguidas.	El sexto intento queda bloqueado temporalmente.
Bovinos	Registrar un arete ya existente.	Se muestra el error de arete duplicado; no se inserta.
Nacimientos	Fecha de la cría anterior a la de la madre.	Se rechaza con mensaje de validación.
Bajas	Dar de baja un bovino y luego intentar vacunarlos.	El bovino ya no aparece en el selector de vacunación.
Vacunación	Fecha de aplicación futura.	Se rechaza con mensaje de validación.
Seguridad	Acceder a un módulo sin sesión iniciada.	Redirige al login.
Seguridad	Usuario con rol sin permiso intenta entrar a un módulo.	Responde 403, página de acceso denegado.

11.2 Pruebas unitarias (siguiente incremento)

Como parte de la buena práctica de pruebas automatizadas, se plantea incorporar pruebas unitarias con PHPUnit en la siguiente iteración del proyecto, priorizando la lógica que no depende directamente de la base de datos:

- `AnimalTest`: validación del método `tieneAreteValido()` con aretes vacíos, válidos y de longitud excedida.
- `SesionTest`: generación y validación del token CSRF (`validarToken` con token correcto, incorrecto y ausente).
- `AutorizacionTest`: verificación de la matriz de permisos por rol (`permite()`) para cada combinación de rol y módulo.

Estas tres clases se probarían de forma aislada (sin conexión a base de datos), por lo que son las candidatas naturales para una primera suite de pruebas unitarias con PHPUnit, ejecutable con `vendor/bin/phpunit tests`.

12. Configuración de los ambientes de desarrollo y pruebas

Elemento	Valor / configuración
Servidor de aplicaciones	Apache (incluido en XAMPP), sirviendo la carpeta htdocs/BovinNet.
Lenguaje	PHP 8, con declare(strict_types=1) en todos los archivos de código.
Base de datos	MySQL / MariaDB (incluido en XAMPP); base de datos "bovinnet".
Administración de base de datos	phpMyAdmin (http://localhost/phpmyadmin), incluido en XAMPP.
Ruta del proyecto	C:\xampp\htdocs\BovinNet (Windows) o equivalente en htdocs/BovinNet.
Punto de entrada	http://localhost/BovinNet/ redirige a public/login.php .
Configuración de conexión	config/db.php (no versionado); plantilla en config/db.example.php.
Scripts de base de datos	sql/bovinnet.sql (esquema completo) y sql/datos_iniciales.sql (rol, finca y usuario administrador inicial).
Editor / IDE	Cualquier editor con soporte de PHP (Visual Studio Code, PHPStorm); no se requieren extensiones específicas más allá del resaltado de sintaxis PHP.
Control de versiones	Git; repositorio local inicializado en la raíz del proyecto (ver sección 13). Se recomienda publicarlo en un servicio remoto (GitHub, GitLab) para su entrega como URL.

13. Control de versiones

El código fuente se gestiona con Git. Se inicializó un repositorio local en la raíz del proyecto y, hasta el momento, tiene dos commits: uno con la versión funcional inicial (módulos de bovinos, nacimientos, bajas, vacunación y seguridad) y uno posterior que reemplazó los datos de ejemplo del panel principal por consultas reales a la base de datos. El archivo config/db.php se excluye del control de versiones mediante .gitignore, ya que contiene las credenciales locales de conexión a la base de datos.

Para publicar el repositorio en un servicio remoto (por ejemplo GitHub) y obtener la URL de entrega solicitada, se sugieren los siguientes pasos:

```
git remote add origin https://github.com/usuario/bovinnet.git
git branch -M main
git push -u origin main
```

A partir de ese primer commit, cada incremento posterior (por ejemplo, la incorporación de las pruebas unitarias planteadas en la sección 11.2) debe registrarse como un nuevo commit, con un mensaje breve que describa el cambio.

14. Conclusiones

- La integración de los módulos de Bovinos, Nacimientos, Bajas y Vacunación sobre una arquitectura por capas común permitió reutilizar el mismo patrón de codificación (Controlador → Repositorio → Modelo → Vista) en cada incremento, reduciendo el esfuerzo de desarrollo de los módulos posteriores.

- La incorporación de la capa de seguridad (autenticación, autorización por rol, protección CSRF y cifrado de contraseñas) como un componente transversal, en lugar de dispersarla en cada controlador, facilita mantenerla y auditarla desde un solo lugar (BaseController y el paquete Security).
- El panel principal, que inicialmente era una maqueta con datos de ejemplo, se conectó a consultas reales sobre las tablas de nacimiento, baja, vacunación y animal. Esto obligó a descartar el indicador de "vacunas pendientes/próximas" original, porque la base de datos no registra un calendario de protocolo sanitario, solo las vacunas ya aplicadas: se optó por no simular ese dato.
- Quedan como trabajo pendiente para una próxima iteración: la implementación del módulo de Reportes con datos reales, la incorporación de pruebas unitarias automatizadas con PHPUnit, y la publicación del repositorio en un servicio remoto para facilitar el trabajo colaborativo.